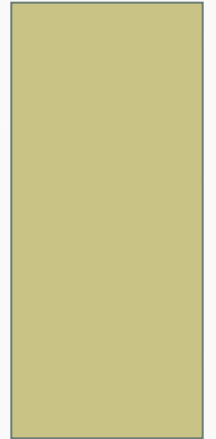


EC-200 DATA STRUCTURES

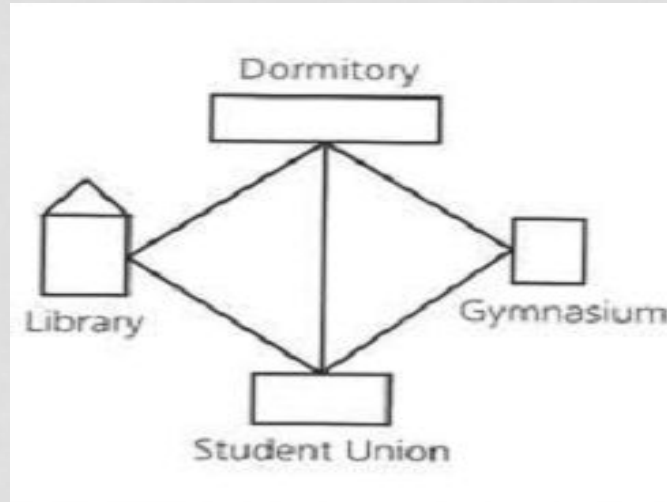
LECTURE 14



GRAPH

- A **graph** is a collection of **nodes** (or **vertices**, singular is **vertex**) and **edges** (or **arcs**)
 - Each node contains an **element**
 - Each edge connects two nodes together (or possibly the same node to itself) and may contain an **edge attribute**
- A **directed graph** is one in which the edges have a direction
- An **undirected graph** is one in which the edges do not have a direction

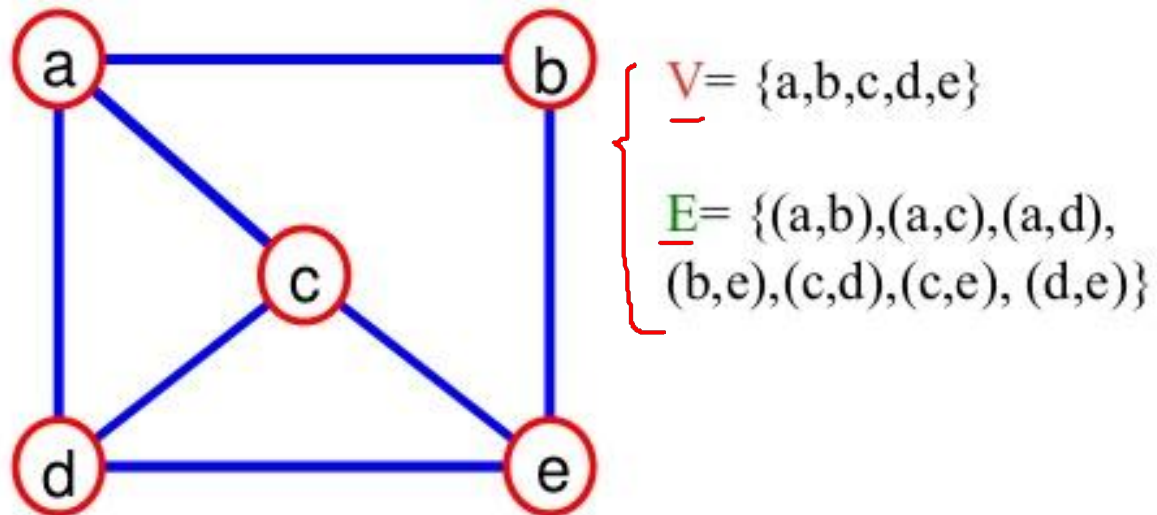
GRAPH



- Graph is a non-linear data structure comprised of two sets:
 - A set of **vertices** V
 - A set of **edges** E
- $G = \{V, E\}$

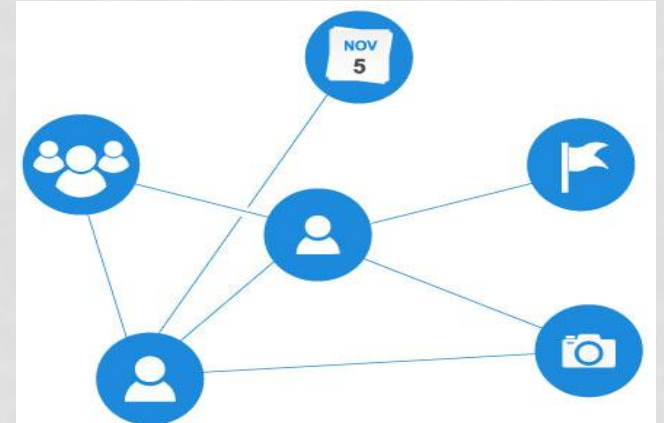
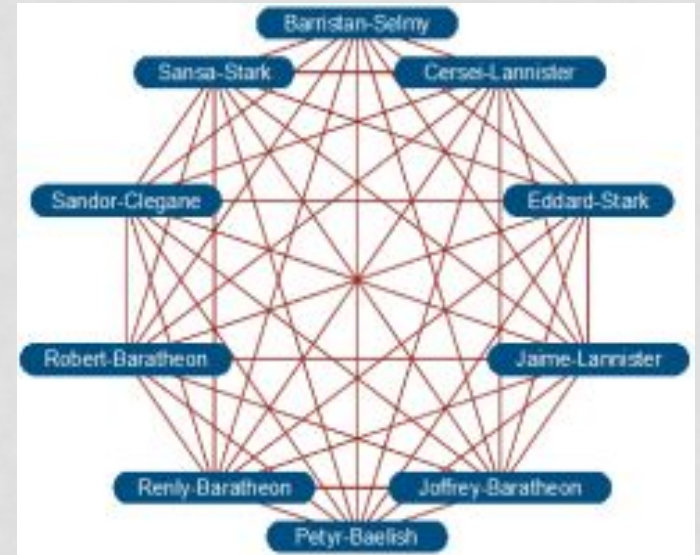
GRAPH

- A graph $G = (\underline{V}, E)$ is composed of:
 - $\underline{V}$: set of **vertices**
 - E : set of **edges** connecting the **vertices** in V
- An **edge** $e = (u, v)$ is a pair of **vertices**
- Example:



APPLICATIONS

- computer networks
- airline flights
- road map ←
- course prerequisite structure
- tasks for completing a job
- flow of control through a program

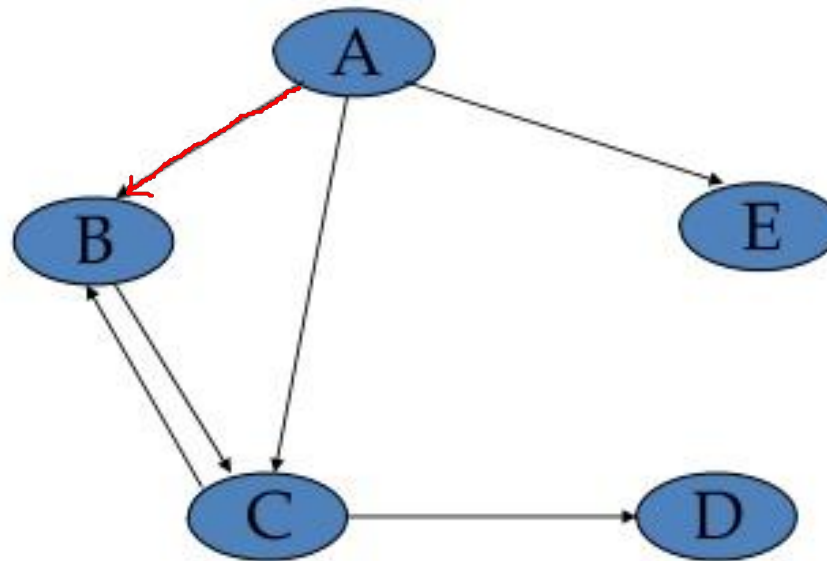


DIRECTED VS UNDIRECTED GRAPH

- An **undirected graph** is one in which the pair of vertices in a edge is unordered, $(v_0, v_1) = (v_1, v_0)$
- A **directed graph** is one in which each edge is a directed pair of vertices, $\langle v_0, v_1 \rangle \neq \langle v_1, v_0 \rangle$



DIRECTED GRAPH

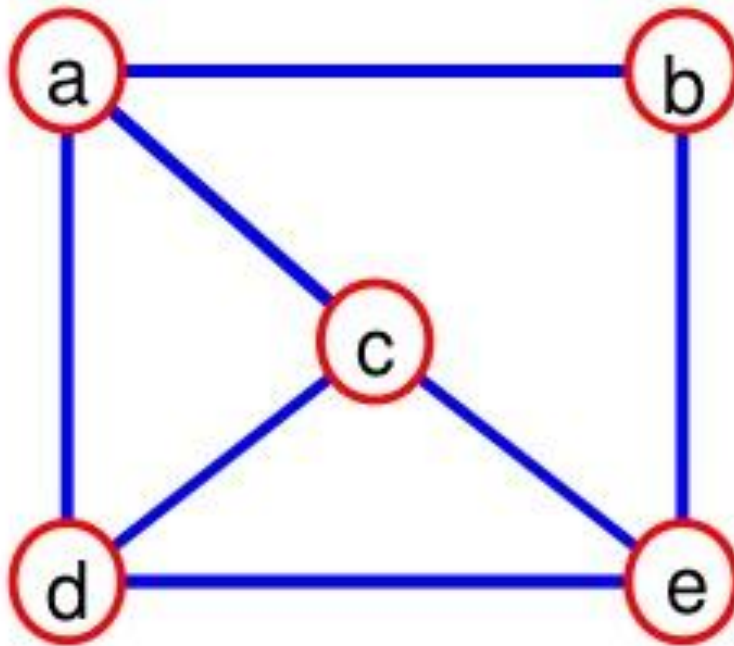


$V = [A, B, C, D, E]$

$E = [\langle A, B \rangle, \langle B, C \rangle, \langle C, B \rangle, \langle A, C \rangle, \langle A, E \rangle, \langle C, D \rangle]$

~~$\langle v_1, v_2 \rangle \langle v_2, v_1 \rangle$~~

UNDIRECTED GRAPH



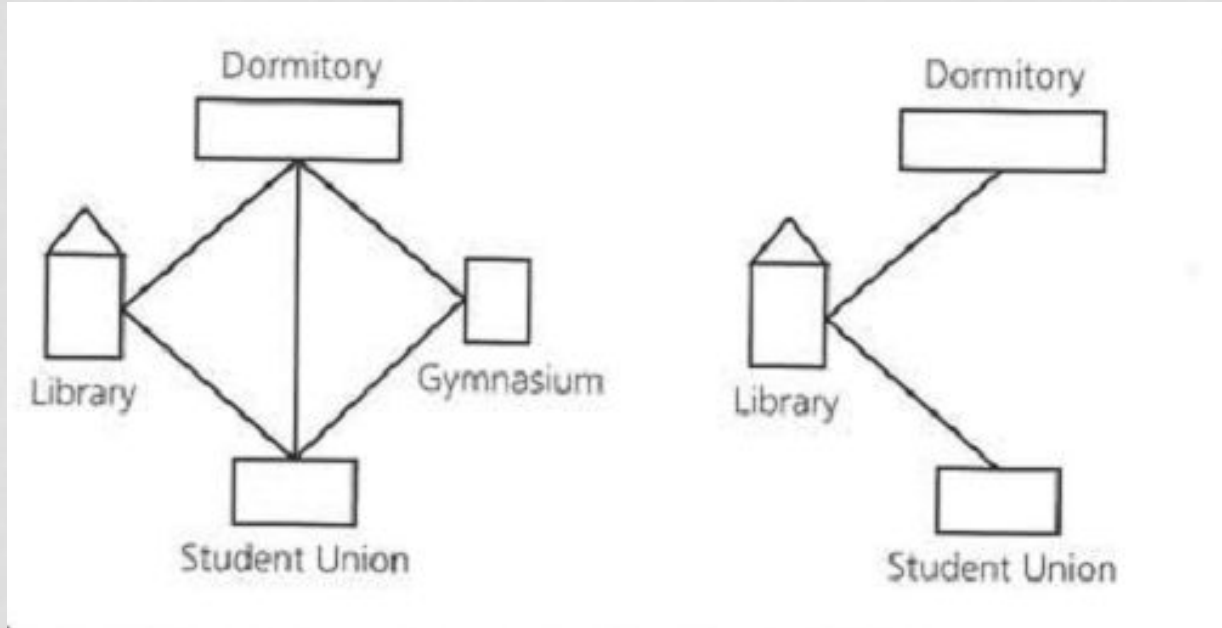
$V = \{a, b, c, d, e\}$

$E = \{(a,b), (a,c), (a,d), (b,e), (c,d), (c,e), (d,e)\}$

TERMINOLOGY

- The **size** of a graph is the number of *nodes* in it
- The **empty graph** has size zero (no nodes)
- If two nodes are connected by an edge, they are **neighbors** (and the nodes are **adjacent** to each other)
- The **degree of a node** is the number of edges it has
- For directed graphs,
 - If a directed edge goes from node *S* to node *D*, we call *S* the **source** and *D* the **destination** of the edge
 - The edge is an **out-edge** of *S* and an **in-edge** of *D*
 - *S* is a **predecessor** of *D*, and *D* is a **successor** of *S*
 - The **in-degree** of a node is the number of in-edges it has
 - The **out-degree** of a node is the number of out-edges it has

TERMINOLOGY



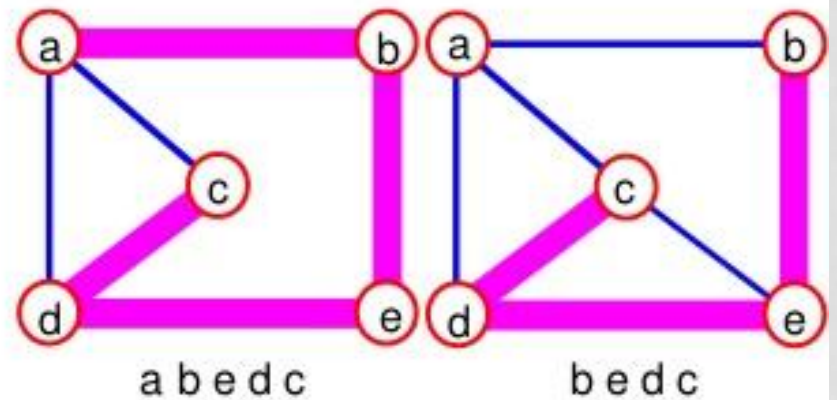
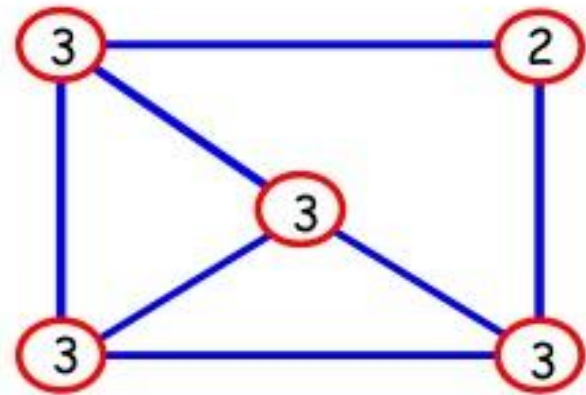
- **Subgraph:** A subset of vertices and edges of the graph
- **Adjacent Vertices:** Two vertices that are joined by an edge

TERMINOLOGY

- Path: A path between two vertices is a sequence of edges
- **Simple Path**: A path in which each vertex appears only once
- **Simple Cycle**: A path that begins and ends at the same vertex. Rest of the vertices appear only once
- **Cycle**: A path that begins and ends at the same vertex.

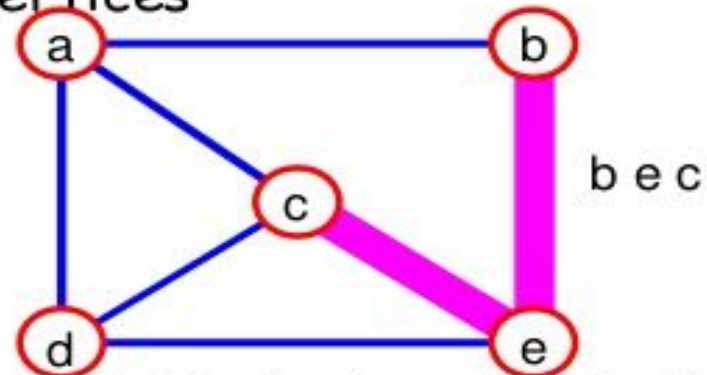
TERMINOLOGY

- **path**: sequence of vertices $v_1, v_2, \dots, v_k$ such that consecutive vertices v_i and v_{i+1} are adjacent.

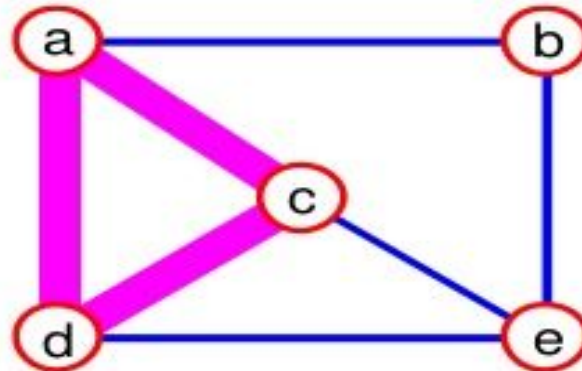


TERMINOLOGY

- **simple path**: no repeated vertices



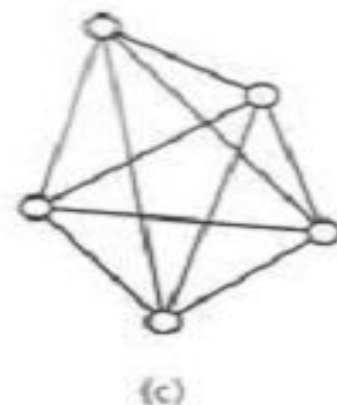
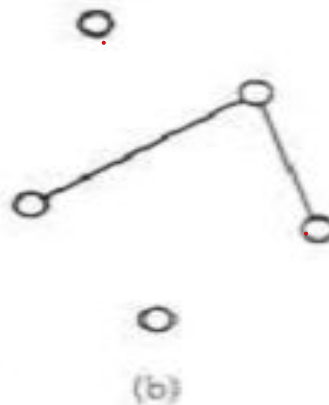
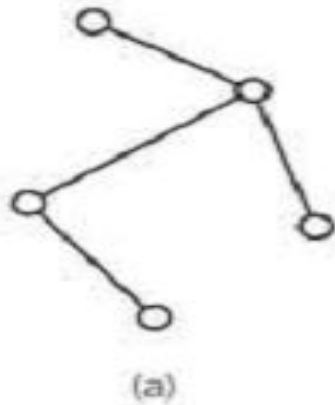
- **cycle**: simple path, except that the last vertex is the same as the first vertex



a c d a



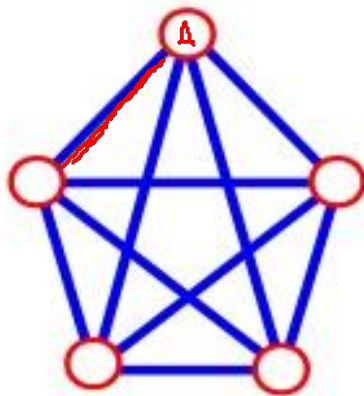
CONNECTED, DISCONNECTED, AND COMPLETE GRAPHS



- A connected graph has a **path** between every pair of distinct vertices
- A complete graph has an **edge** between every pair of distinct vertices

COMPLETE GRAPH

- Let $\underline{n}$ = #vertices, and $\underline{m}$ = #edges
- **A complete graph**: one in which all pairs of vertices are adjacent
- *How many total edges in a complete graph?*
 - Each of the n vertices is incident to $n-1$ edges, however, we would have counted each edge twice! Therefore, intuitively, $m = n(n-1)/2$.
- Therefore, if a graph is not complete, $m < \underline{n(n-1)/2}$



$$\begin{aligned} n &= 5 \\ m &= (5 * 4)/2 = 10 \end{aligned}$$

$$\frac{n * (n-1)}{2}$$

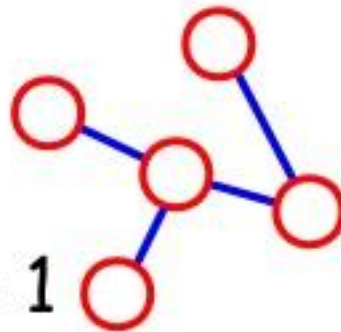
CONNECTED GRAPH

n = #vertices

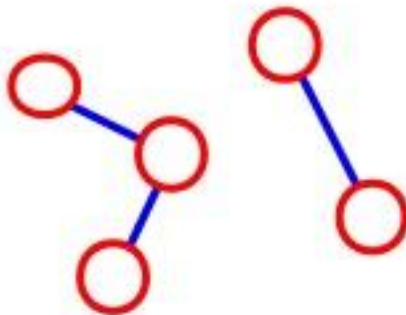
m = #edges

- For a tree $m = \underline{n - 1}$

If $m < \underline{n - 1}$, G is not connected



$n = 5$
 $m = 4$



$n = 5$
 $m = 3$

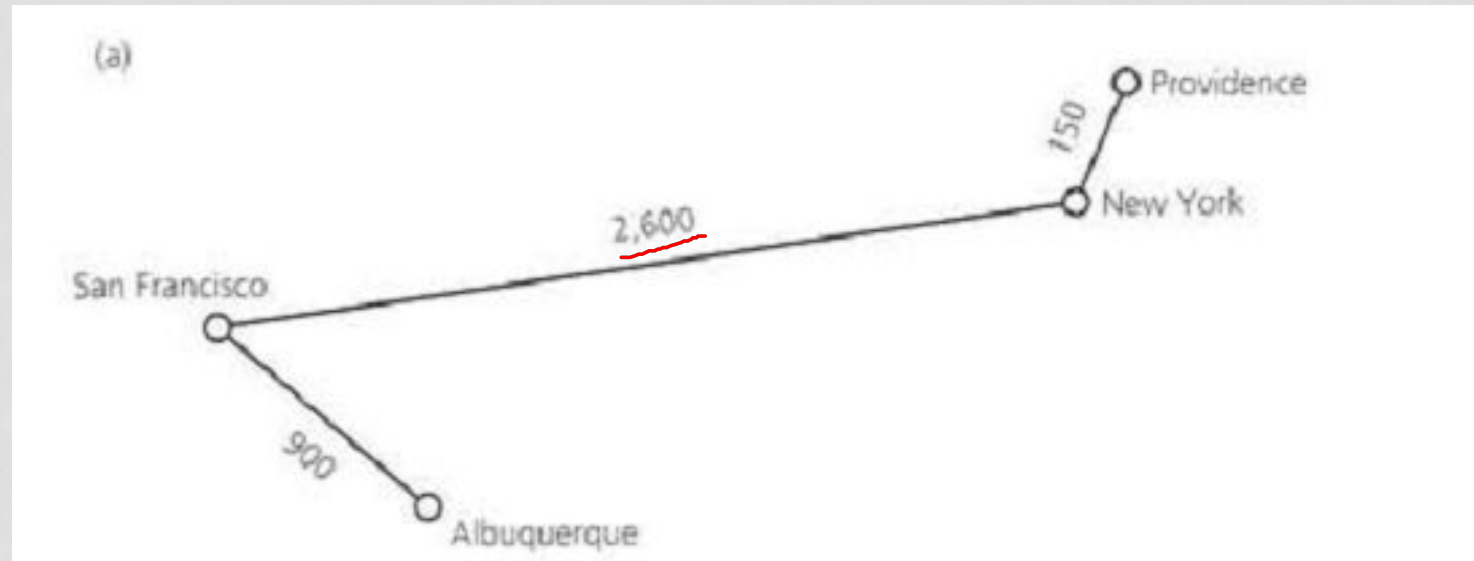


TERMINOLOGY



- An undirected graph is **connected** if there is a path from every node to every other node
- A *directed graph* is **strongly connected** if there is a path from every node to every other node
- A directed graph is **weakly connected** if the underlying undirected graph is connected
- Node **X** is **reachable** from node **Y** if there is a path from **Y** to **X**
- A subset of the nodes of the graph is a **connected component** (or just a **component**) if there is a path from every node in the subset to every other node in the subset

WEIGHTED GRAPH



- The edges of a weighted graph have numeric labels

IMPLEMENTING GRAPHS

- ✓ • **Adjacency Matrix**
 - Adjacency matrix of a graph with n vertices $0, 1, \dots, n-1$ is an $n \times n$ array matrix such that $\text{matrix}[i][j]$ is 1 (true) if there is an edge from i to j , and is 0 (false) otherwise.
- ✓ • **Adjacency List**
 - Adjacency list of a graph with n vertices $0, 1, \dots, n-1$ consists of n linked lists. The i th linked list contains a node for vertex j if and only if the graph contains an edge from i to j

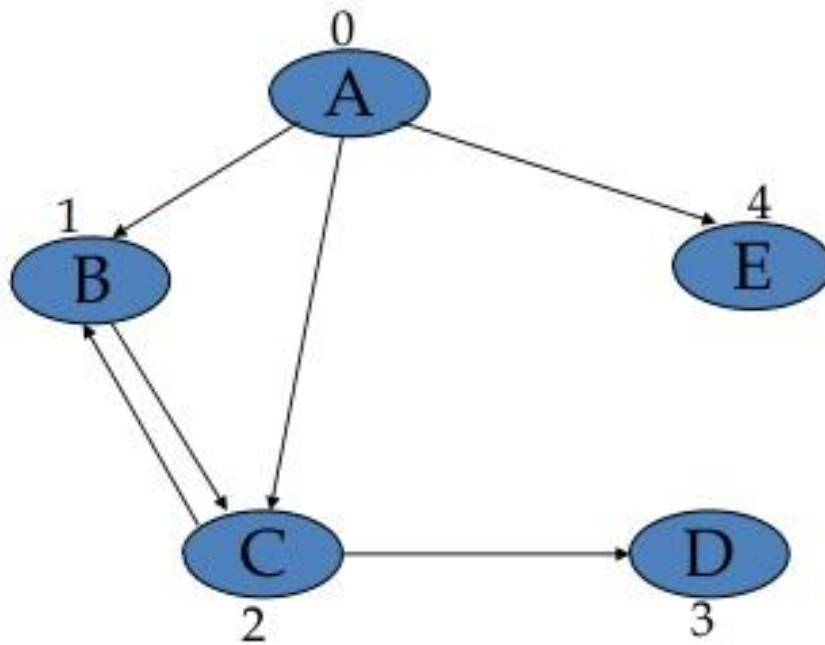
IMPLEMENTATION

- storing the vertices
 - each vertex has a unique identifier and, maybe, other information
 - for efficiency, associate each vertex with a number that can be used as an index
- storing the edges
 - adjacency matrix - represent all possible edges
 - adjacency lists - represent only the existing edges

REPRESENTING VERTEX

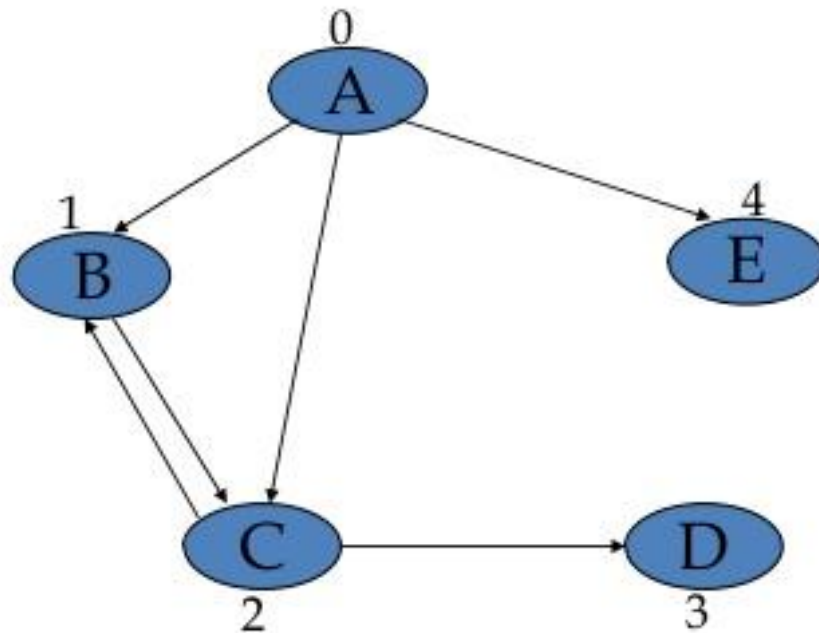
- when a vertex is added to the graph, assign it a number
 - vertices are numbered between 0 and $n-1$
- graph operations start by looking up the number associated with a vertex
- many data structures to use
 - any of the associative data structures
 - for small graphs a vector can be used
 - search will be $O(n)$

VERTEX VECTOR



0	A
1	B
2	C
3	D
4	E

ADJACENCY MATRIX

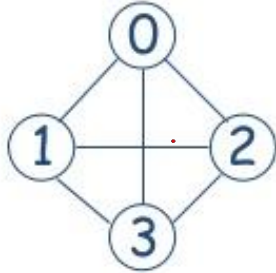


0	A
1	B
2	C
3	D
4	E

$n \times n$

	A	B	C	D	E
	0	1	2	3	4
A → 0	0	1	1	0	1
B 1	0	0	1	0	0
C 2	0	1	0	1	0
D 3	0	0	0	0	0
E 4	0	0	0	0	0

ADJACENCY MATRIX



G_1

$$\begin{matrix}
 & \begin{matrix} 0 & 1 & 2 & 3 \end{matrix} \\
 \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}
 \end{matrix}$$

symmetric

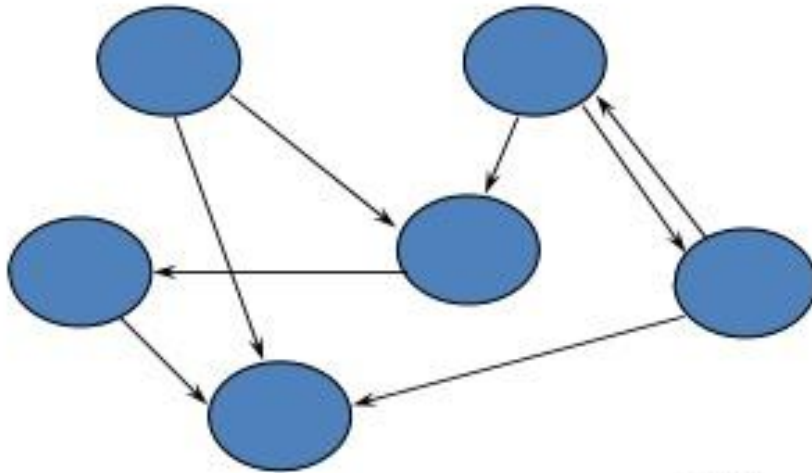


G_2

$$\begin{matrix}
 & \begin{matrix} 0 & 1 & 2 \end{matrix} \\
 \begin{matrix} 0 \\ 1 \\ 2 \end{matrix} & \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}
 \end{matrix}$$

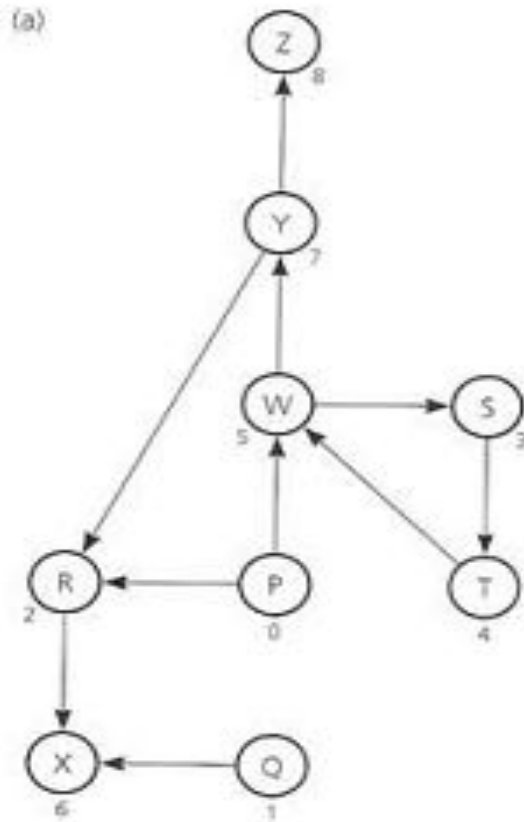
Asymmetric

SPACE COMPLEXITY?



a V^2 matrix is needed for
a graph with V vertices

ADJACENCY MATRIX OF A DIRECTED (UNWEIGHTED) GRAPH



(b)

		0	1	2	3	4	5	6	7	8
		P	Q	R	S	T	W	X	Y	Z
0	P	0	0	1	0	0	1	0	0	0
1	Q	0	0	0	0	0	0	1	0	0
2	R	0	0	0	0	0	0	1	0	0
3	S	0	0	0	0	1	0	0	0	0
4	T	0	0	0	0	0	1	0	0	0
5	W	0	0	0	1	0	0	0	1	0
6	X	0	0	0	0	0	0	0	0	0
7	Y	0	0	1	0	0	0	0	0	1
8	Z	0	0	0	0	0	0	0	0	0

FIGURE 13-6

(a) A directed graph and (b) its adjacency matrix

ADJACENCY MATRIX OF A DIRECTED (WEIGHTED)GRAPH

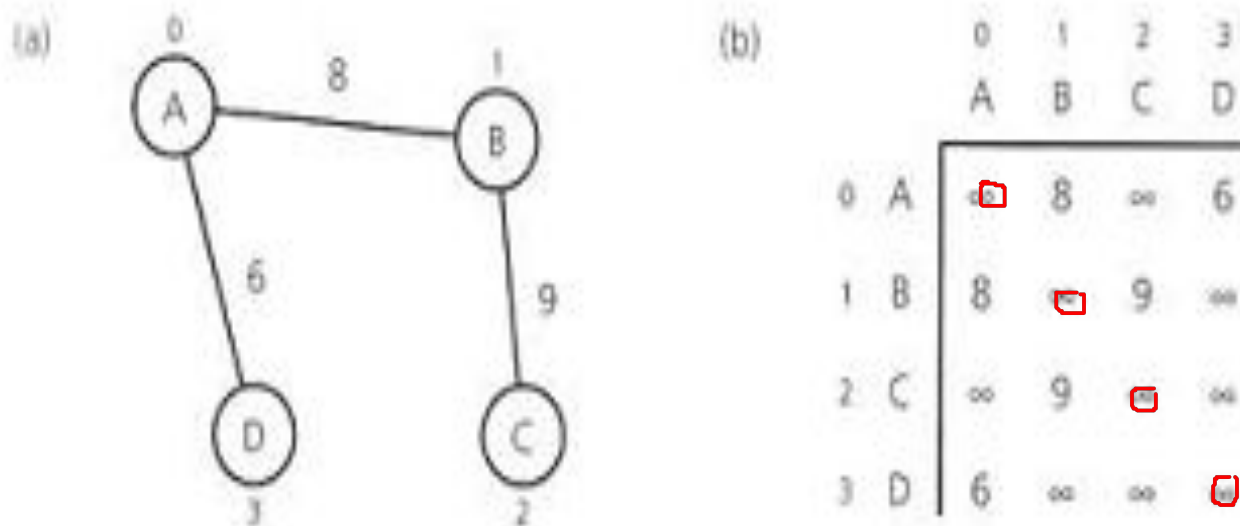
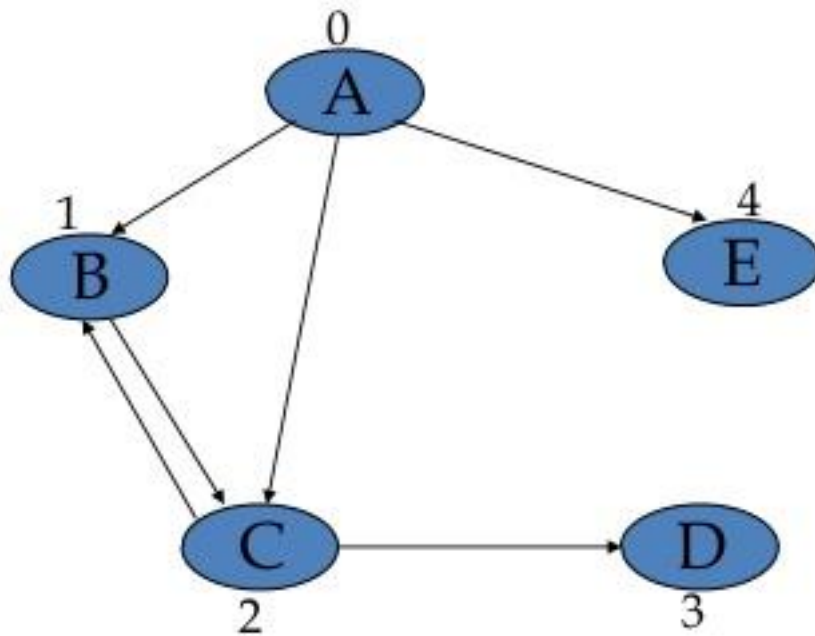


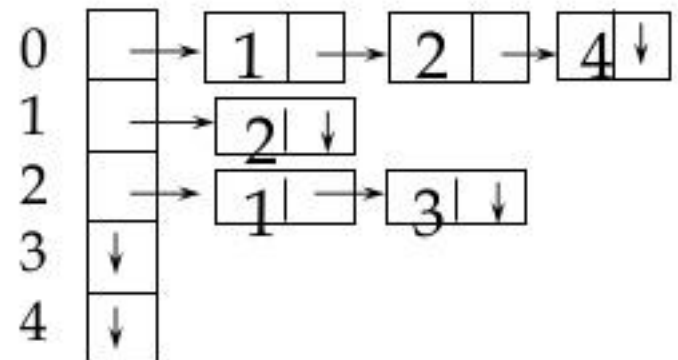
FIGURE 13-7

(a) A weighted undirected graph and (b) its adjacency matrix

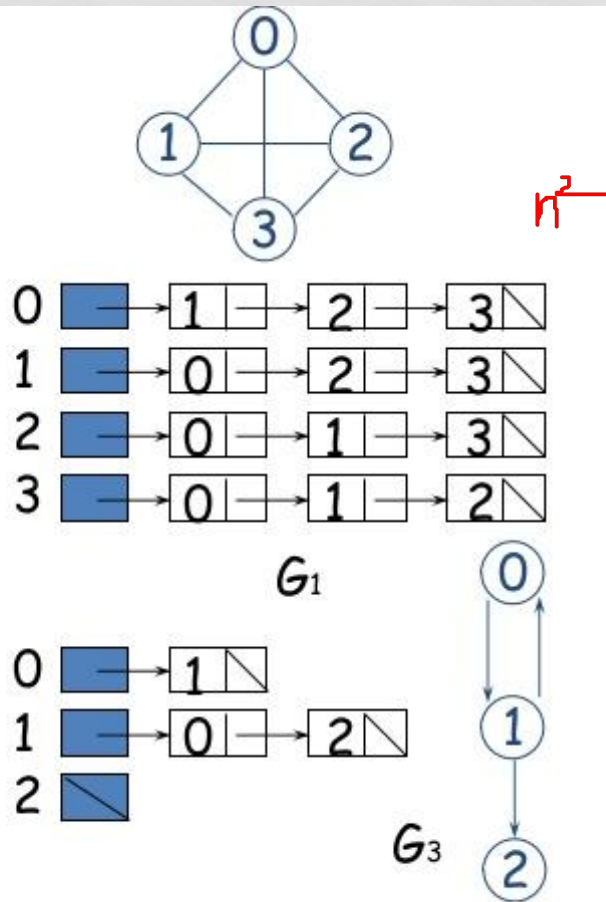
ADJACENCY LIST



0	A
1	B
2	C
3	D
4	E



ADJACENCY LIST



ADJACENCY LIST OF A DIRECTED (UNWEIGHTED)GRAPH

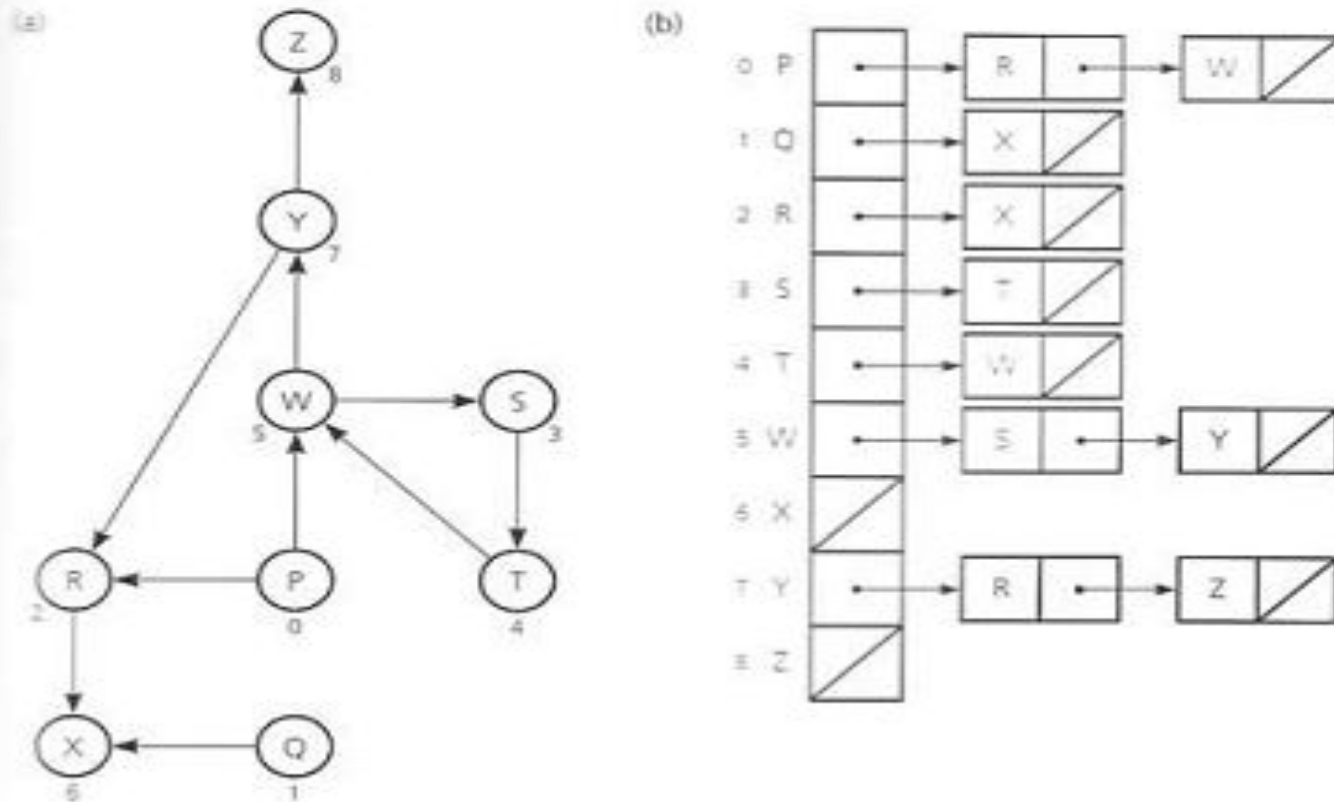


FIGURE 13-8

(a) A directed graph and (b) its adjacency list

READING

- Chapter 20, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- <https://leapgraph.com/graph-data-structures-applications>